Chapter 2: Modeling

Youngseo Cho

2025-12-31

Contents

1 Introduction **1**

2 Polynomial Interpolation **1**
2.1 Equi-spaced vs Chebyshev Nodes 1

3 Two-dimensional Design Points **3**

4 Radial Basis Interpolation **4**

5 Ordinary Kriging **5**

6 Gaussian Process **6**
6.1 Prior, Data, and Posterior Distribution 6
6.2 Gaussian Process with Fitted Parameters 7

7 Polynomial Regression with Noise **8**
7.1 3rd and 8th Degree Polynomial (Equi-spaced vs D-optimal) 8

8 Gaussian Process Regression Comparison **11**

9 Summary **12**

1 Introduction

This chapter covers various modeling approaches including polynomial interpolation, radial basis functions, Kriging, and Gaussian processes.

Citation: V. R. Joseph (2026). *Experimental Design for Data Science and Engineering*. CRC Press.

2 Polynomial Interpolation

2.1 Equi-spaced vs Chebyshev Nodes

```
f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
test <- seq(0, 1, length = 301)
true <- f(test)
n <- 10

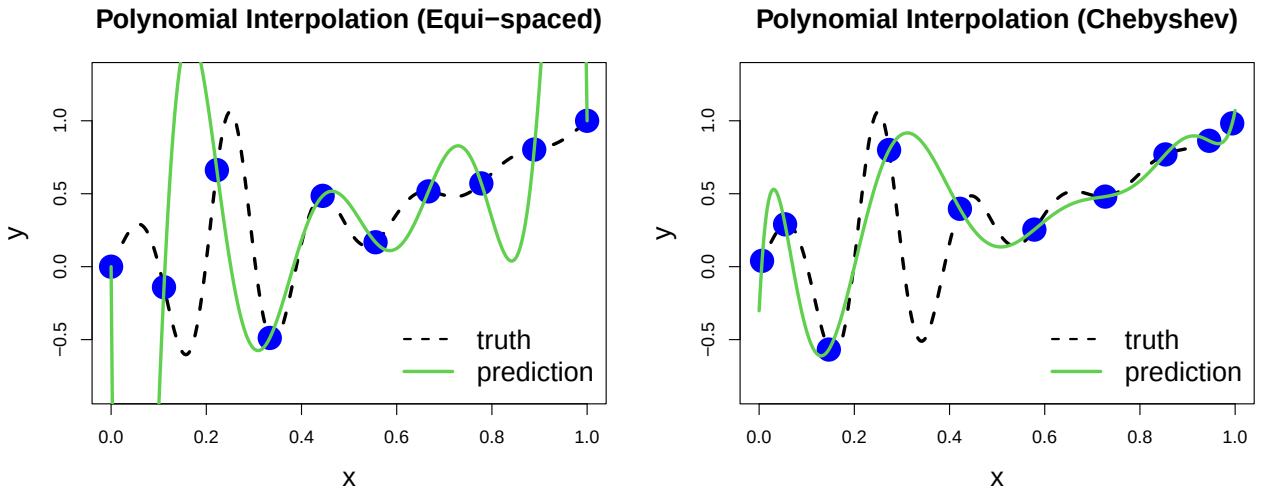
par(mfrow = c(1, 2))
```

```r
# Equi-spaced design
D <- ((1:n) - 1) / (n - 1)
y <- f(D)
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "Polynomial Interpolation (Equi-spaced)", cex.main = 1.5, cex.lab = 1.5)
points(cbind(D, y), pch = 16, cex = 3, col = "blue")

d <- 2 * D - 1
X <- NULL
for (i in 1:n) X <- cbind(X, cos((i - 1) * acos(d)))
a <- solve(X, y)
u <- seq(0, 1, length = 301)
yhat <- u
us <- 2 * u - 1
for (i in 1:301) {
  yhat[i] <- 0
  for (j in 1:n) yhat[i] <- yhat[i] + a[j] * cos((j - 1) * acos(us[i]))
}
lines(u, yhat, col = 3, lwd = 3)
legend("bottomright", legend = c("truth", "prediction"), lty = c(2, 1),
       col = c(1, 3), lwd = c(2, 3), bty = "n", cex = 1.5)

# Chebyshev nodes
d <- cos((2 * (1:n) - 1) / (2 * n) * pi)
D <- (d + 1) / 2
y <- f(D)
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "Polynomial Interpolation (Chebyshev)", cex.main = 1.5, cex.lab = 1.5)
points(cbind(D, y), pch = 16, cex = 3, col = "blue")

X <- NULL
for (i in 1:n) X <- cbind(X, cos((i - 1) * acos(d)))
a <- solve(X, y)
for (i in 1:301) {
  yhat[i] <- 0
  for (j in 1:n) yhat[i] <- yhat[i] + a[j] * cos((j - 1) * acos(us[i]))
}
lines(u, yhat, col = 3, lwd = 3)
legend("bottomright", legend = c("truth", "prediction"), lty = c(2, 1),
       col = c(1, 3), lwd = c(2, 3), bty = "n", cex = 1.5)
```



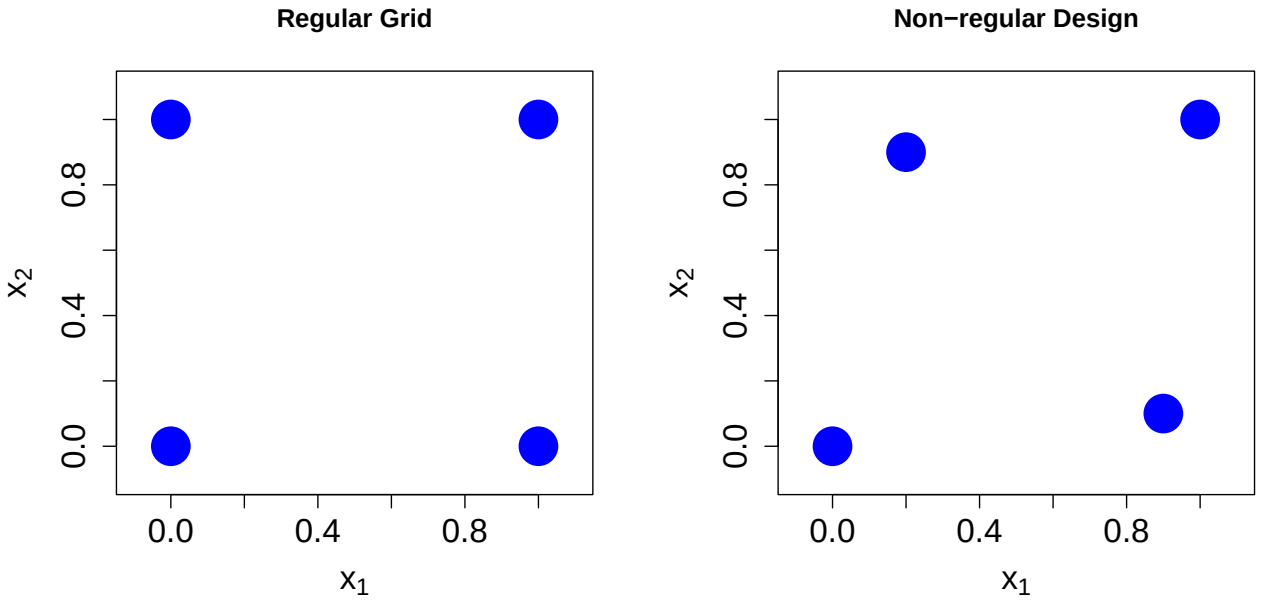

```
par(mfrow = c(1, 1))
```

3 Two-dimensional Design Points

```
par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 2))

# Regular grid
D <- matrix(c(0, 0, 0, 1, 1, 0, 1, 1), nrow = 4, ncol = 2, byrow = TRUE)
plot(D, pch = 16, cex = 4, col = "blue", xlim = c(-0.1, 1.1), ylim = c(-0.1, 1.1),
     xlab = expression(x[1]), ylab = expression(x[2]), cex.axis = 1.5, cex.lab = 1.5,
     main = "Regular Grid")

# Non-regular design
D <- matrix(c(0, 0, 0.2, 0.9, 0.9, 0.1, 1, 1), nrow = 4, ncol = 2, byrow = TRUE)
plot(D, pch = 16, cex = 4, col = "blue", xlim = c(-0.1, 1.1), ylim = c(-0.1, 1.1),
     xlab = expression(x[1]), ylab = expression(x[2]), cex.axis = 1.5, cex.lab = 1.5,
     main = "Non-regular Design")
```



```r
par(mfrow = c(1, 1))
```

4 Radial Basis Interpolation

```r
f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
test <- seq(0, 1, length = 301)
true <- f(test)
n <- 10
D <- ((1:n) - 1) / (n - 1)
y <- f(D)

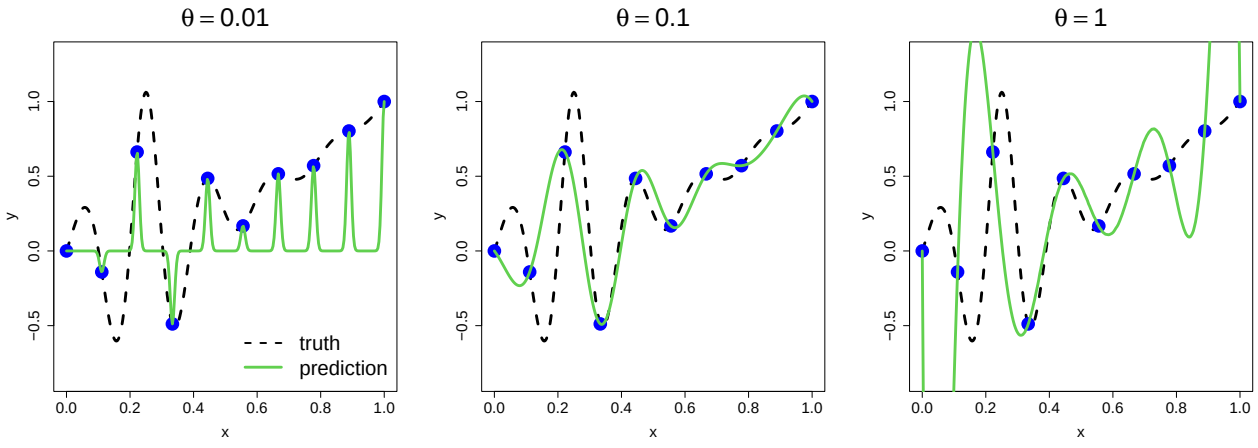
par(mar = c(5, 5, 4, 2))
par(mfrow = c(1, 3))

E <- as.matrix(dist(D))

for (theta in c(0.01, 0.1, 1)) {
  plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
       ylim = c(min(true) - 0.25, max(true) + 0.25),
       main = bquote(theta == .(theta)), cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
  points(cbind(D, y), pch = 16, cex = 3, col = "blue")

  R <- exp(-(E / theta)^2)
  coef <- solve(R, y)
  basis <- function(h) exp(-(h / theta)^2)
  r <- function(x) basis(x - D)
  fhat <- function(x) sum(r(x) * coef)
  pred <- sapply(test, fhat)
  lines(test, pred, col = 3, lwd = 3)

  if (theta == 0.01) {
    legend("bottomright", legend = c("truth", "prediction"), lty = c(2, 1),
           col = c(1, 3), lwd = c(2, 3), bty = "n", cex = 2)
  }
}
```




```r
par(mfrow = c(1, 1))
```

5 Ordinary Kriging

```
f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
test <- seq(0, 1, length = 301)
true <- f(test)
n <- 10
D <- ((1:n) - 1) / (n - 1)
y <- f(D)

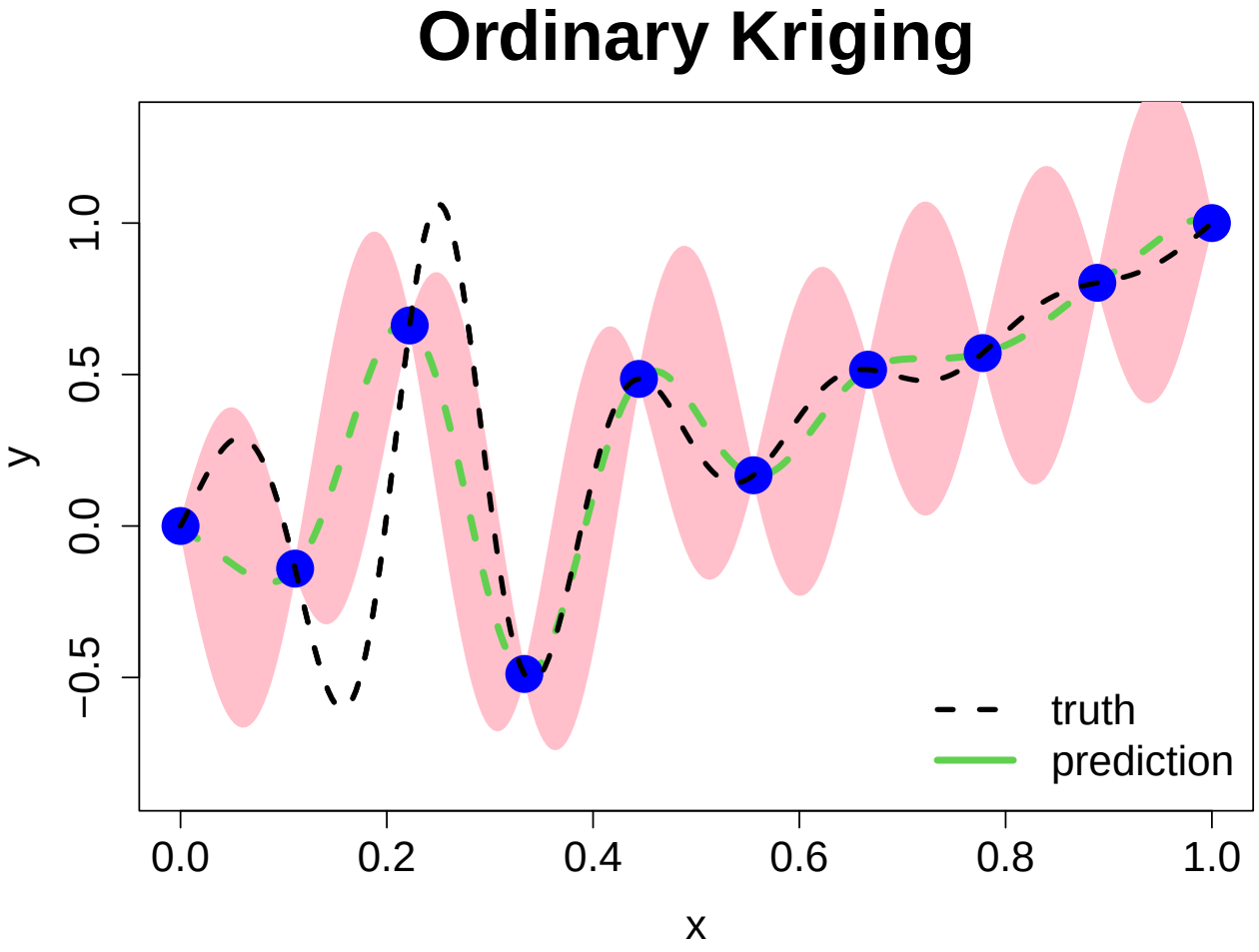
par(mar = c(5, 5, 4, 2))
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "Ordinary Kriging", cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)

E <- as.matrix(dist(D))
theta <- 0.08
R <- exp(-(E / theta)^2)
basis <- function(h) exp(-(h / theta)^2)
r <- function(x) basis(x - D)
one <- rep(1, n)
Rinv <- solve(R + 10^(-10) * diag(n))
mu <- drop(t(one) %*% Rinv %*% y / (t(one) %*% Rinv %*% one))
coef <- Rinv %*% (y - mu)
tau2 <- drop(1/n * t(y - mu) %*% Rinv %*% (y - mu))

f.var <- function(x) {
  fac <- 1 / (t(one) %*% Rinv %*% one)
  val <- tau2 * (1 - t(r(x)) %*% Rinv %*% r(x) + fac * (1 - t(r(x)) %*% Rinv %*% one)^2)
  return(val)
}

fhat <- function(x) mu + t(r(x)) %*% coef
N <- 301
pred <- low <- up <- numeric(N)
for (i in 1:N) {
  pred[i] <- fhat(test[i])
  SD <- sqrt(f.var(test[i]))
  low[i] <- pred[i] - 2 * SD
  up[i] <- pred[i] + 2 * SD
}

polygon(c(test, rev(test)), c(low, rev(up)), col = "pink", border = NA, lwd = 2)
lines(test, low, lwd = 2, col = "pink")
lines(test, up, lwd = 2, col = "pink")
lines(test, pred, col = 3, lwd = 4, lty = 2)
points(cbind(D, y), pch = 16, cex = 3, col = "blue")
lines(test, true, lty = 2, lwd = 3)
legend("bottomright", legend = c("truth", "prediction"), lty = c(2, 1),
       col = c(1, 3), lwd = c(3, 4), bty = "n", cex = 1.5)
```



6 Gaussian Process

6.1 Prior, Data, and Posterior Distribution

```
library(rkriging)
library(MASS)
library(pdist)
library(Matrix)

f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
N <- 301
test <- seq(0, 1, length = N)
true <- f(test)
n <- 10
D <- ((1:n) - 1) / (n - 1)
y <- f(D)

kernel <- Get.Kernel(0.08 / sqrt(2), type = "Gaussian")
a <- Fit.Kriging(D, y, fit = FALSE, kernel = kernel)
tau2 <- Get.Kriging.Parameters(a)$nu2
pred <- Predict.Kriging(a, test)

set.seed(1)
```

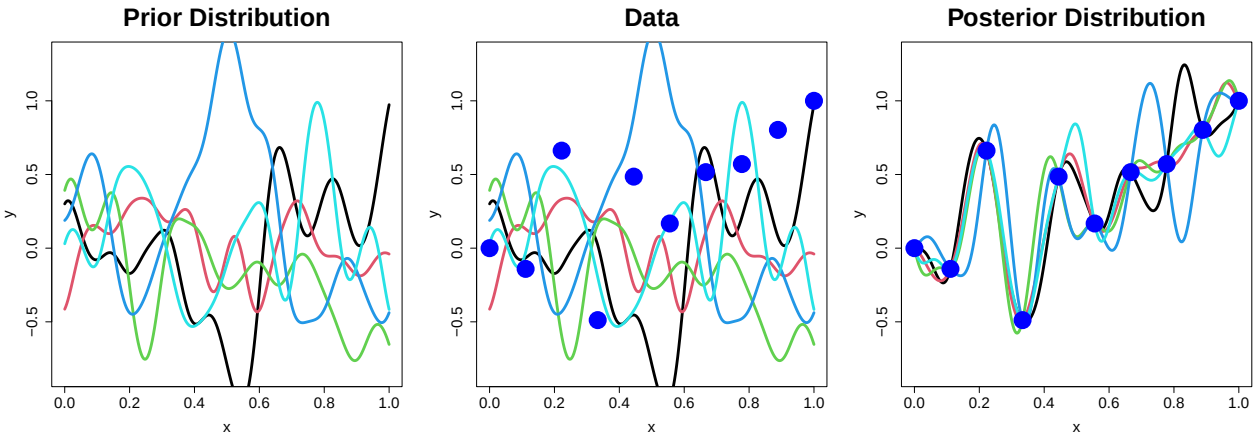
```r
S0 <- Evaluate.Kernel(kernel, test)
u <- mvrnorm(n = 5, mu = rep(0, N), Sigma = tau2 * S0)

par(mfrow = c(1, 3))

# Prior distribution
matplot(test, t(u), type = "l", xlab = "x", ylab = "y", col = 1:5, lty = 1, lwd = 3,
        ylim = c(min(true) - 0.25, max(true) + 0.25),
        main = "Prior Distribution", cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)

# Data
matplot(test, t(u), type = "l", xlab = "x", ylab = "y", col = 1:5, lty = 1, lwd = 3,
        ylim = c(min(true) - 0.25, max(true) + 0.25),
        main = "Data", cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
points(cbind(D, y), pch = 16, cex = 4, col = "blue")

# Posterior distribution
R <- Evaluate.Kernel(kernel, D)
L <- chol(R + 10^(-10) * diag(n))
M <- t(as.matrix(pdist(cbind(D), cbind(test))))
M <- exp(-M^2 / 0.08^2)
b <- forwardsolve(t(L), t(M))
S <- tau2 * (S0 - t(b) %*% b)
S <- nearPD(S)$mat
me <- pred$mean
set.seed(1)
u <- mvrnorm(n = 5, mu = me, Sigma = S)
matplot(test, t(u), type = "l", xlab = "x", ylab = "y", col = 1:5, lty = 1, lwd = 3,
        ylim = c(min(true) - 0.25, max(true) + 0.25),
        main = "Posterior Distribution", cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
points(cbind(D, y), pch = 16, cex = 4, col = "blue")
```



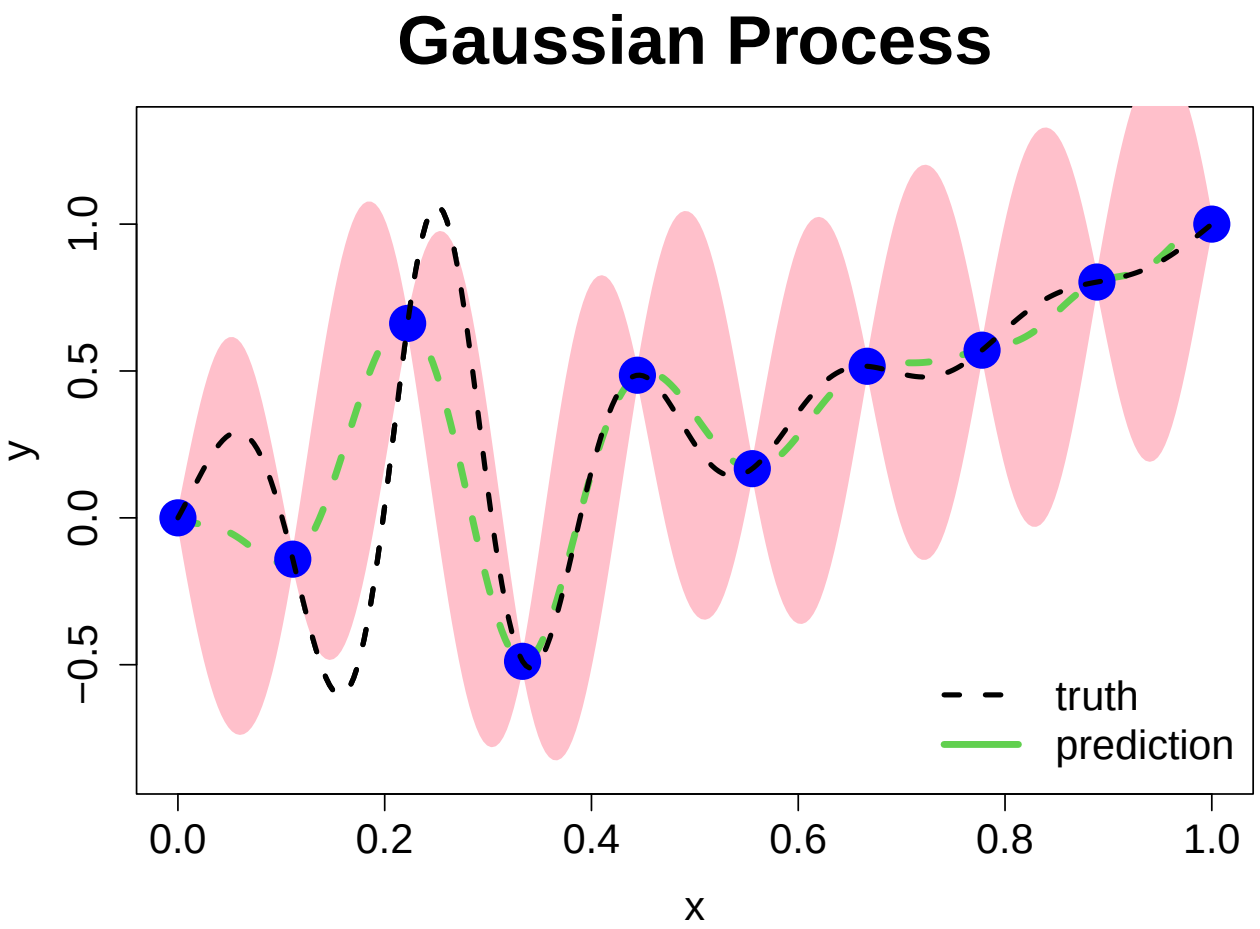

```r
par(mfrow = c(1, 1))
```

6.2 Gaussian Process with Fitted Parameters

```r
a <- Fit.Kriging(D, y, kernel.parameters = list(type = "Gaussian"))
theta <- Get.Kriging.Parameters(a)$lengthscale * sqrt(2)
pred <- Predict.Kriging(a, test)
```

```r
low <- pred$mean - 2 * pred$sd
up <- pred$mean + 2 * pred$sd

plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "Gaussian Process", cex.main = 2.5, cex.axis = 1.5, cex.lab = 1.5)
polygon(c(test, rev(test)), c(low, rev(up)), col = "pink", border = NA, lwd = 2)
lines(test, low, lwd = 2, col = "pink")
lines(test, up, lwd = 2, col = "pink")
lines(test, pred$mean, col = 3, lwd = 4, lty = 2)
points(cbind(D, y), pch = 16, cex = 3, col = "blue")
lines(test, true, lty = 2, lwd = 3)
legend("bottomright", legend = c("truth", "prediction"), lty = c(2, 1),
       col = c(1, 3), lwd = c(3, 4), bty = "n", cex = 1.5)
```



7 Polynomial Regression with Noise

7.1 3rd and 8th Degree Polynomial (Equi-spaced vs D-optimal)

```r
library(AlgDesign)

f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
test <- seq(0, 1, length = 301)
```

```r
true <- f(test)

set.seed(5)
n <- 10
r <- 2

# Equi-spaced design
D <- ((1:n) - 1) / (n - 1)
D <- rep(D, r)
e <- rnorm(n * r, sd = 0.1)
y <- f(D) + e

par(mar = c(5, 5, 4, 2))
par(mfrow = c(2, 2))

# 3rd degree - equi-spaced
d <- 2 * D - 1
a <- lm(y ~ poly(d, 3))
u <- seq(0, 1, length = 301)
us <- 2 * u - 1
pred <- predict(a, newdata = data.frame(d = us), interval = "confidence")
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "3rd Degree (Equi-spaced)", cex.main = 1.5, cex.lab = 1.5)
polygon(c(u, rev(u)), c(pred[, 2], rev(pred[, 3])), col = "pink", border = NA)
lines(u, pred[, 1], col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
lines(u, f(u), lty = 2, lwd = 3)

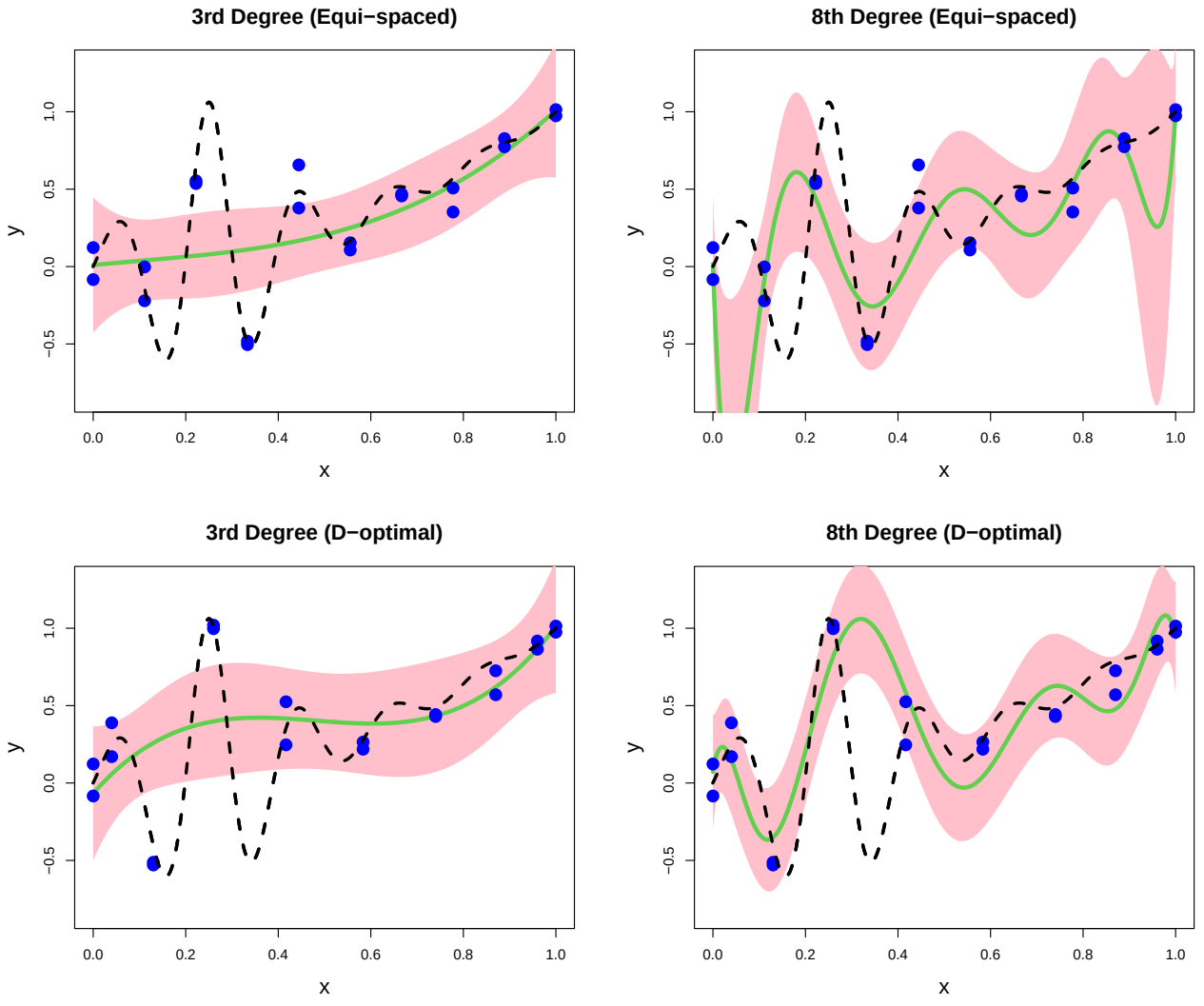
# 8th degree - equi-spaced
a <- lm(y ~ poly(d, 8))
pred <- predict(a, newdata = data.frame(d = us), interval = "confidence")
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "8th Degree (Equi-spaced)", cex.main = 1.5, cex.lab = 1.5)
polygon(c(u, rev(u)), c(pred[, 2], rev(pred[, 3])), col = "pink", border = NA)
lines(u, pred[, 1], col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
lines(u, f(u), lty = 2, lwd = 3)

# D-optimal design
cand <- data.frame(x = seq(-1, 1, length = 301))
a_opt <- optFederov(~ 1 + x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6) + I(x^7) + I(x^8) + I(x^9),
                    nTrials = n, data = cand, approximate = FALSE)
D <- rep(c(a_opt$design[, 1]), r)
D <- (D + 1) / 2
y <- f(D) + e

# 3rd degree - D-optimal
d <- 2 * D - 1
a <- lm(y ~ poly(d, 3))
pred <- predict(a, newdata = data.frame(d = us), interval = "confidence")
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
```

```
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "3rd Degree (D-optimal)", cex.main = 1.5, cex.lab = 1.5)
polygon(c(u, rev(u)), c(pred[, 2], rev(pred[, 3])), col = "pink", border = NA)
lines(u, pred[, 1], col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
lines(u, f(u), lty = 2, lwd = 3)

# 8th degree - D-optimal
a <- lm(y ~ poly(d, 8))
pred <- predict(a, newdata = data.frame(d = us), interval = "confidence")
plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "8th Degree (D-optimal)", cex.main = 1.5, cex.lab = 1.5)
polygon(c(u, rev(u)), c(pred[, 2], rev(pred[, 3])), col = "pink", border = NA)
lines(u, pred[, 1], col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
lines(u, f(u), lty = 2, lwd = 3)
```




```
par(mfrow = c(1, 1))
```

8 Gaussian Process Regression Comparison

```r
library(rkriging)
library(AlgDesign)

f <- function(x) sin(10 * pi * x) / (1 + 64 * (x - 0.25)^2) + x^2
test <- seq(0, 1, length = 301)
true <- f(test)

set.seed(5)
n <- 10
re <- 2

par(mfrow = c(1, 2))

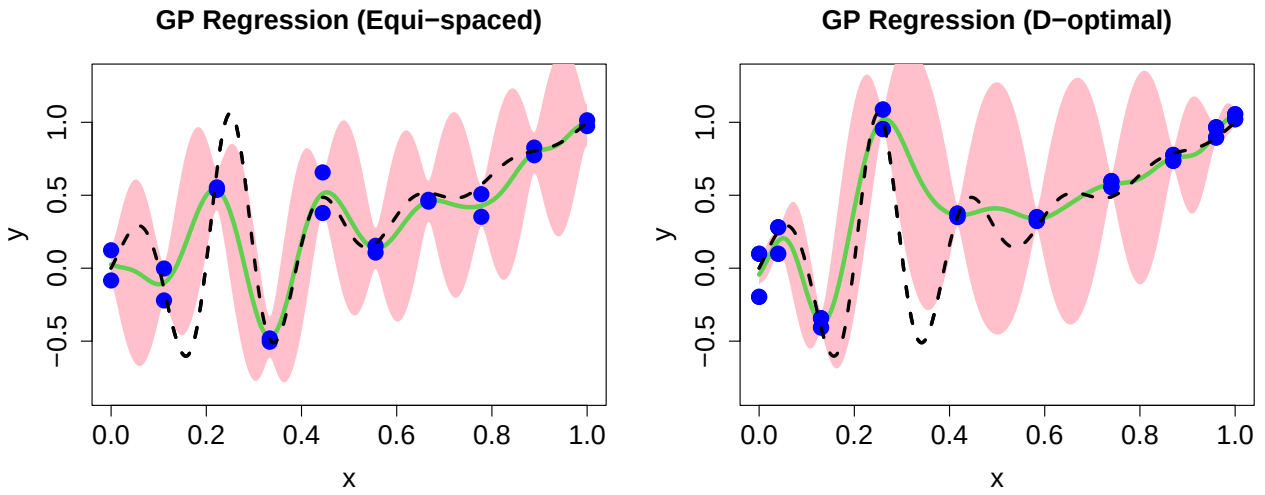
# Equi-spaced design
D <- ((1:n) - 1) / (n - 1)
D <- rep(D, re)
e <- rnorm(n * re, sd = 0.1)
y <- f(D) + e
a <- Fit.Kriging(D, y, interpolation = FALSE, kernel.parameters = list(type = "Gaussian"))
pred <- Predict.Kriging(a, test)
low <- pred$mean - 2 * pred$sd
up <- pred$mean + 2 * pred$sd

plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "GP Regression (Equi-spaced)", cex.main = 1.5, cex.axis = 1.5, cex.lab = 1.5)
polygon(c(test, rev(test)), c(low, rev(up)), col = "pink", border = NA, lwd = 2)
lines(test, pred$mean, col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
lines(test, true, lty = 2, lwd = 3)

# D-optimal design
cand <- data.frame(x = seq(-1, 1, length = 301))
a_opt <- optFederov(~ 1 + x + I(x^2) + I(x^3) + I(x^4) + I(x^5) + I(x^6) + I(x^7) + I(x^8) + I(x^9),
                    nTrials = n, data = cand, approximate = FALSE)
D <- rep(c(a_opt$design[, 1]), re)
D <- (D + 1) / 2
D <- rep(D, re)
e <- rnorm(n * re, sd = 0.1)
y <- f(D) + e
a <- Fit.Kriging(D, y, interpolation = FALSE, kernel.parameters = list(type = "Gaussian"))
pred <- Predict.Kriging(a, test)
low <- pred$mean - 2 * pred$sd
up <- pred$mean + 2 * pred$sd

plot(test, true, type = "l", xlab = "x", ylab = "y", lwd = 3, lty = 2, col = 1,
     ylim = c(min(true) - 0.25, max(true) + 0.25),
     main = "GP Regression (D-optimal)", cex.main = 1.5, cex.axis = 1.5, cex.lab = 1.5)
polygon(c(test, rev(test)), c(low, rev(up)), col = "pink", border = NA, lwd = 2)
lines(test, pred$mean, col = 3, lwd = 4)
points(cbind(D, y), pch = 16, cex = 2, col = "blue")
```

```
lines(test, true, lty = 2, lwd = 3)
```




```
par(mfrow = c(1, 1))
```

9 Summary

This chapter covered:

1. **Polynomial interpolation** with equi-spaced and Chebyshev nodes
2. **Radial basis function interpolation** with different bandwidth parameters
3. **Ordinary Kriging** for spatial interpolation
4. **Gaussian processes** - prior, data conditioning, and posterior distributions
5. **Polynomial regression** comparing equi-spaced and D-optimal designs
6. **Kernel ridge regression** and **GP regression** for noisy data